Rethinking Spreadsheets

Gavin Lowe

June 9, 2026

1 Introduction

The goal of the *Rethinking Spreadsheets* project is to reconsider the interface that existing spreadsheets provide to the user, and to improve upon it.

The traditional way to program a spreadsheet is to type a formula into a cell; the formula represents a calculation based on the values in other cells. The cell then displays the value of the formula. Often, the user copies the formula in one cell into adjacent cells; the application adjusts references to other cells appropriately.

The difficulty with this approach is that it is very hard to scrutinise the code. The user has to click on each cell individually and read the formula. If is very easy to make mistakes.

For example, the Economists Reinhart and Rogoff claimed[1] that if a country has national debt of over 90% of GDP, it leads to much reduced growth. Various nations adopted policies of austerity based on this paper. However, Herdon, Ash and Pollin[2] found that the result was incorrect, partly because of an error in the spreadsheet which omitted some countries. Such errors are easy to make, but hard to spot.

Our thesis is that there should be a separation between the rules that define calculations and the results of those calculations. The rules are given in a separate script; and the results appear in the spreadsheet.

In order to allow scrutiny of the script, and to support maintenance, it should be a reasonable length. If similar calculations are performed to produce multiple cells, the relevant formula should be defined once, and then

[1] Carmen M. Reinhart and Kenneth S. Rogoff, "Growth in a Time of Debt". American Economic Review. 100 (2): 573–78, 2010.

[2] Thomas Herndon, Michael Ash, Robert Pollin. "Does High Public Debt Consistently Stifle Economic Growth? A Critique of Reinhart and Rogoff". Political Economy Research Institute, University of Massachusetts Amherst. 2013.

applied to relevant cells via an iteration: it should not be necessary to copy the formula.

Note, though, that we propose keeping input data and calculated cells as part of the same spreadsheet: it is often convenient to view those together.

Further, the language used for most existing spreadsheets—simple functions that are combined together—is not convenient for anything other than very simple applications. Spreadsheet users should be able to use a language similar to modern programming languages. The script language we propose is a declarative language, influenced by both Haskell and the functional fragment of Scala. In particular, the language allows type checking of the script.

A script contains a combination of declarations (of values and functions), and commands (e.g. to write a value into a cell).

In Section 2 we present a simple example script, to introduce the main parts of the language. In Section 3 we describe how to use the spreadsheet application. Then in Section 4 we present a more detailed description of the language. We assume some familiarity with programming.

2 A simple example script

Figure 1 gives a simple (and somewhat contrived) example script, intended to introduce the language. The script expects to find integers in an initial segment of column A. It calculates their factorials, and writes them in column B. It also prints the total of those factorials below.

We describe the script below. The script is executed in order.

The characters "//" turn the rest of the line into a comment. Block comments (maybe spanning multiple lines, and maybe nested) can be written between "/*" and "*/".

Line 2 defines two values In and Out, representing the input and output columns. It is a good idea to give names to relevant rows and columns, in the same way that it is a good idea to give names to relevant constants in standard programming languages. A column value can be defined by preceding the corresponding letter with "#", so, for example, "#A" represents column A. Likewise a row value can be defined by preceding the row number with "#", for example "#0". We call these forms *row* and *column literals*.

Line 5 defines the factorial function. Definitions of functions are introduced using the keyword **def**. Each parameter of a function is given a type. The return type of a function should be given when the function is recursive—which is the case for all three functions in this script—or when a reference is made to the function before its definition (e.g. when two functions are defined by mutual recursion). The **assert** statement is an assertion, capturing

```
1  // The input and output columns.
2  val In = #A; val Out = #B
3
4  /* The factorial of n. */
5  def fact(n: Int): Int = { assert(n >= 0); if(n <= 1) 1 else n*fact(n−1) }
6
7  /* The sum of xs. */
8  def sum(xs: List[Int]): Int = if(xs == []) 0 else head(xs) + sum(tail(xs))
9
10 /* Row for the first empty cell in column c from row r. */
11 def firstEmptyInCol(c: Column, r: Row): Row =
12   Cell(c,r) match{ case Empty => r; case n: Int => firstEmptyInCol(c, r+1) }
13
14 // Row for the first empty cell in In
15 val firstEmpty = firstEmptyInCol(In, #0)
16
17 // Write factorials of entries in In into Out.
18 for(r <− #0 until firstEmpty) Cell(Out,r) = fact(Cell(In,r))
19
20 // Write the total.
21 Cell(In, firstEmpty+1) = "Total:"
22 Cell(Out, firstEmpty+1) = sum([Cell(Out,r): Int | r <− #0 until firstEmpty])
```

Figure 1: A simple example script.

the natural precondition of the factorial function. The rest of the definition of fact is a straightforward recursion, whose meaning should be obvious.

The sum function at line 8 calculates the sum of a list of Ints. If A is a type, then List[A] is the type of lists that contain values of type A. The value [] is the empty list. More generally, a list can be defined by writing its elements between square brackets, separated by commas, for example "[2, 4, 6, 8]". The function head returns the first element of a nonempty list; and the function tail returns the list containing all except the first element of a nonempty list. Again, the definition is a straightforward recursion.

The expression Cell(c,r) represents the value in the spreadsheet cell in column c and row r. In the case where c and r are column and row literals, this can be written more concisely. For example, #B3 is shorthand for Cell(#B,#3). The script may make assumptions about the types of values that it reads from cells; if a cell does not contain the expected type, an error is generated.

The function firstEmptyInCol(c, r), at line 11, finds the first empty cell in

column c, from row r onwards, and returns the relevant row. Column is the type of columns, and Row is the type of rows. The function requires the cell read to be either empty or an integer. This is checked using pattern matching on the value, using syntax similar to that in Scala. Each case in a pattern matching expression is of the form **case** *pat* => *e*, where *pat* is a pattern, and *e* is an expression giving the result when that pattern is matched. The pattern Empty matches an empty cell. Hence, if Cell(c,r) is empty, firstEmptyInCol returns r. The pattern n: Int matches an integer, and binds the name n to that integer in the subsequent expression. Hence, if Cell(c,r) contains an integer, the firstEmptyInCol function recurses with firstEmptyInCol(c, r+1), where r+1 represents the row after r; thus, the function simply continues searching from the next row. If the value read from the cell does not match any of the patterns—in this case, if it is not empty or an integer—the pattern matching fails, and the spreadsheet application gives an error message.

Line 15 sets firstEmpty to be the first row in column In that contains an empty cell.

A cell can be written using a command of the form "Cell(c,r) = *e*", where *e* defines the value written. For example, line 21 writes the string "Total:" into the cell below the first empty cell in column In.

Line 18 writes into column Out the factorials of values in corresponding cells in column In. It does this using a **for** loop. A **for** loop of the form **for**(x <− xs) *block* executes *block* for each value of x from the list xs. Here *block* is a block containing declarations and commands. The expression #0 **until** firstEmpty represents the list of rows from 0, inclusive, until firstEmpty, exclusive, i.e. the rows that were previously found to contain integers in column In. For each such row r, line 18 writes the corresponding factorial into column Out.

Finally, line 22 calculates the sum of the factorials, and writes it into the appropriate cell. The expression in square brackets on the right is a list comprehension, using Haskell-style notation. It forms the list of values in row r of column Out, for each value of r in #0 **until** firstEmpty, i.e. the factorials written at line 18. The subexpression Cell(Out, r): Int tells the spreadsheet application that the cell should contain an integer; the type checker makes use of this information.

3 Using the spreadsheet application

The spreadsheet application can be run by running Scala on the class `spreadsheet.SpreadsheetApp`, providing the name of the script to use. The application uses the Scala Swing library, so the jar file must be on the class-

path. I use the following command (except all on one line):

```
scala -cp .:scala-swing_2.13-3.0.0.jar
  spreadsheet.SpreadsheetApp factorials.dir
```

Optionally, the name of a CSV file can also be provided, giving values to load into the spreadsheet.

Values can be entered into the spreadsheet in the normal way. The script is run automatically each time the user enters a value. Figure 2 gives a screenshot of the window after the user has input values into the first three cells in column A. Cells with a purple background are user inputs; cells with a green background contain values written by the script, with the currently selected cell in a darker shade.

Update screenshots

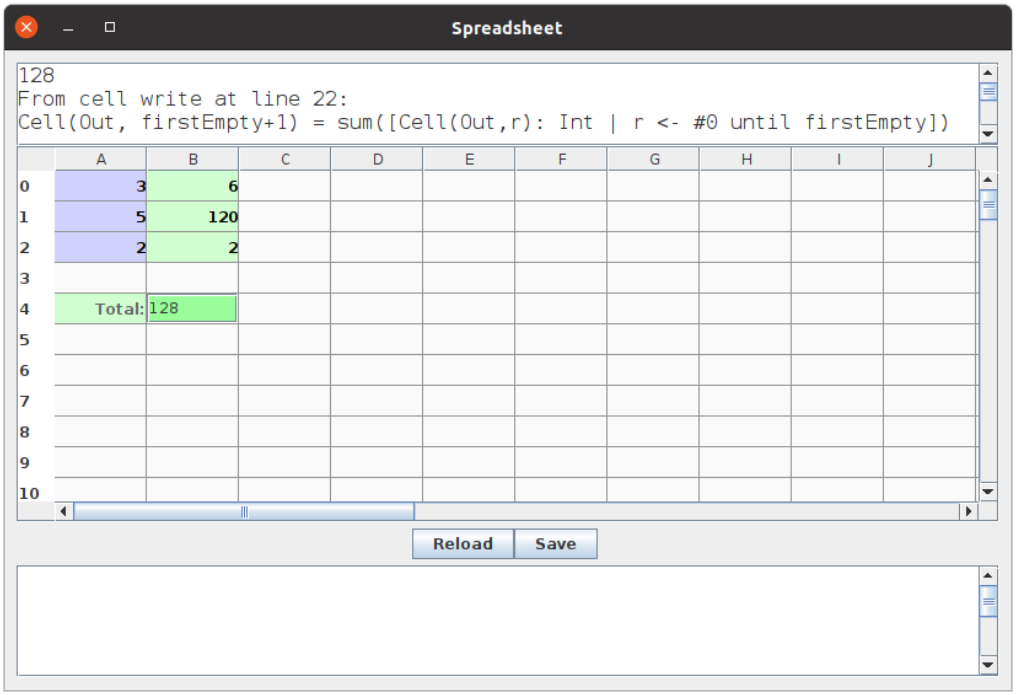

Figure 2: The spreadsheet application window.

The top panel in the window, known as the *selection panel*. gives information about the selected cell. Here it shows that the cell contains the value 128, that the value came from a cell write at line 22; it also gives the corresponding code.

As with any programming language, errors can occur, which can be detected either when the script is type checked, or when the script is executed.

As an example of a type error, suppose we change line 5 of the script to the following:

```
def fact(n: Int): Int = if(n <= 1.0) 1 else n*fact(n-1)
```

This is a type error, because it compares the Int n with the floating point number 1.0. Figure 3 shows what happens when the script is re-loaded (via the "Reload" button). The bottom panel, known as the *error panel*, gives information about each error. Here it indicates that the script expected an Int but found a Float at line 5, and that the erroneous term was 1.0; it also gives enclosing code at various levels.

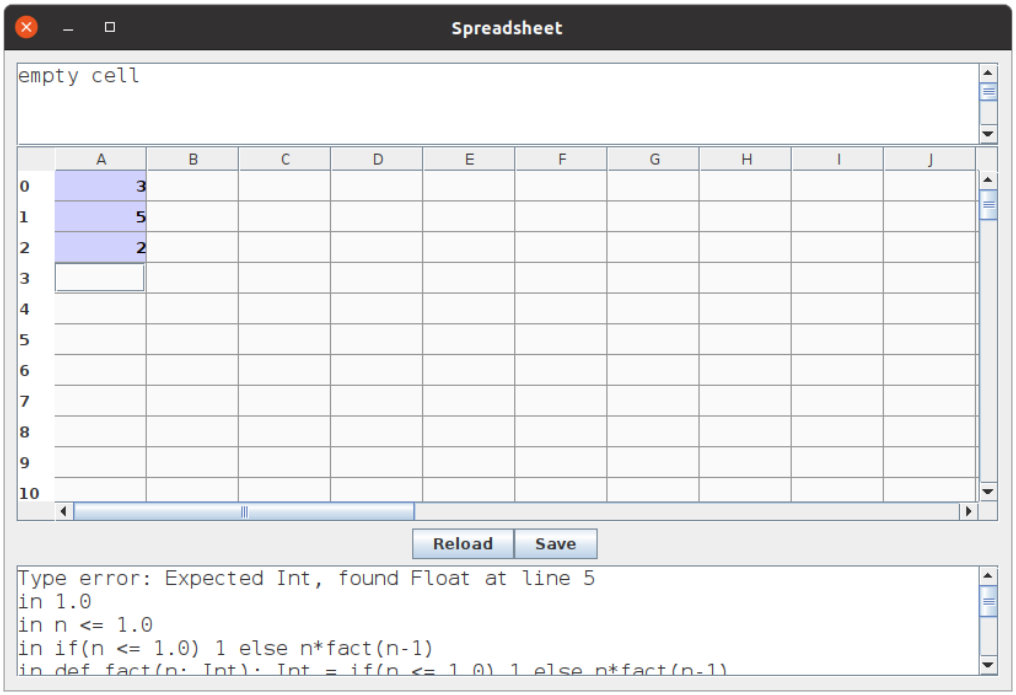

Figure 3: The spreadsheet application indicating a type error.

Execution-time errors include errors common in other programming languages, such as division by zero, or taking the head of an empty list.

A non-standard form of error occurs when the script reads a value from a cell that is not of the expected type. For example, suppose we enter the string three into cell #A0; then the error panel displays the following error:

```
Cannot match String in cell #A0 at line 12
in "Cell(c,r) match{ case Empty => r; case _: Int => firstEmptyInCol(c, r+1) }"
in "firstEmptyInCol(In, #0)"
in "val firstEmpty = firstEmptyInCol(In, #0)"
```

This shows that the pattern matching at line 12 fails when given a string.

Similar errors can occur outside pattern matching. Suppose we change the definition of firstEmptyInCol as follows.

```
def firstEmptyInCol(c: Column, r: Row): Row =
  Cell(c,r) match{ case Empty => r; case _ => firstEmptyInCol(c, r+1) }
```

The pattern "_" matches any value, so the pattern matching will succeed. However, now other errors occur, as illustrated in Figure 4, where cells with a pink background represent errors. The error panel indicates that the cell #A0 read at line 18 found a String when an Int was expected. The same information is included in the selection panel, since cell #B0 is selected. The bottom of the error panel also explains an error corresponding to the write of cell #B4 (this requires scrolling to read fully).

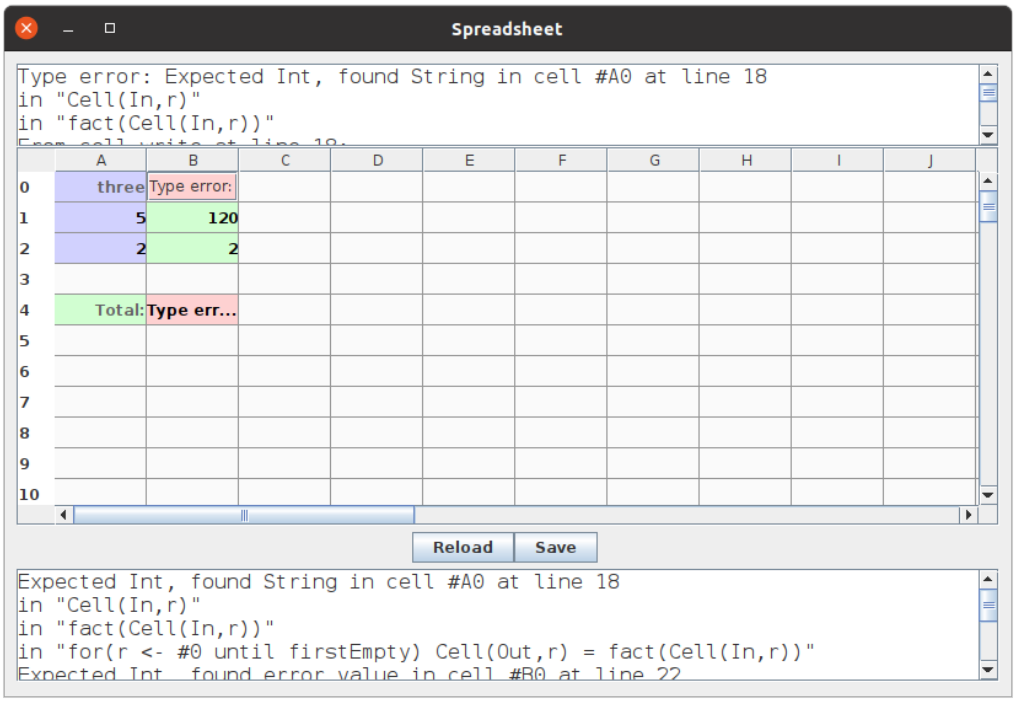

Figure 4: The spreadsheet application indicating a cell read error.

The application partitions cells into *input cells*, where the user has provided input, *output cells*, written to by the script, and *empty cells*. Any attempt to write to an input cell is considered an error and blocked: this prevents an erroneous script from overwriting user data. Likewise, an attempt to write to an output cell for a second time is considered an error.

The "Save" button on the spreadsheet can be used to store the contents of input cells in a CSV file.

4 Script language

In this section we give a more detailed description of the script language. We describe the types in Section 4.1. We describe expressions in Section 4.2. We describe declarations of values and functions in Section 4.3. We then describe cell write commands in Section 4.5, and **for** loop iterations in Section 4.6.

4.1 Types

The script language contains the standard types Int of (32-bit) integers, Float of (32-bit) floating point numbers, Boolean of booleans (with values true and false), and String of strings.

These four types (and no others) can be entered in cells of the spreadsheet. The application decides the type of each value based on how it is entered, mostly in the obvious way; for example, if a value looks like an integer, it is given the type Int. In most cases, a String can be entered by simply typing the text, or can (optionally) be enclosed in quotation marks. However, in the case that a String is required that looks like a value of another type, it *must* be entered inside quotation marks (e.g. "3"). A literal quotation mark can be entered as \" (e.g. "He said \"Hello\"").

The types Row and Column represent rows and columns, respectively.

The type Unit is a type that contains a single value, called the unit value, denoted (). This is the return type for functions that do not return a proper result, but (typically) just perform some calculations and write into cells.

If T is a type, then List[T] represents the type of lists whose elements are from T.

If T1 and T2 are types, then (T1,T2) is the type of pairs (x, y) with $x \in$ T1 and $y \in$ T2. This extends in the obvious way to tuples of size up to four.

If T1 and T2 are types, then T1 => T2 is the type of functions from T1 to T2. Note that the constructor "=>" associates to the right, so T1 => T2 => T3 means T1 => (T2 => T3).

The language uses type classes in a style similar to Haskell. Each type class contains types with certain operators. Type classes can be used to restrict functions to apply only to certain types; see Section 4.3. The current type classes are:

Eq: Equality types, that allow comparison using == and !=; these types include Int, Float, Boolean, String, Row, Column, Unit, List[T] for T an

equality type, and (T$_1$,...,T$_n$) for T$_1$,..., T$_n$ equality types.

Ord: Ordered types, that allow order comparison using <=, <, >= and >; these types include Int, Float, Boolean, String, Row, Column, Unit, List[T] for T an ordered type, and (T$_1$,...,T$_n$) for T$_1$,..., T$_n$ ordered types.

CellType: The types of values that can appear in cells: Int, Float, Boolean, and String.

Each cell type (from CellType) is also an ordered type (Ord) and an equality type (Eq). Each ordered type is also an equality type (the converse is true, but the compiler does not recognise this fact, and it may not remain true in future versions.)

4.2 Expressions

4.2.1 Standard expressions

The script language contains the normal arithmetic operators, +, −, * and /, over Int and Float (over Int, / performs rounding), and % over Int. The language also contains the normal equality (==) and inequality (!=) operators, which can be applied to values of equality types, and the order (<, <=, >, >=) operators, which can be applied to values of ordered types. Note that Int and Float are distinct types, so can't (currently) be combined using these operators. Unary negation can be written using −; note that the expression might need to be included in parentheses when the − could be interpreted as a binary operator, for example,

```
{ val x = 2
  (−1)
}
```

The built-in functions toFloat: Int => Float and toInt: Float => Int can be used to convert between these types.

The language includes the normal boolean binary operators, && and ||. Boolean negation is written using the function not.

Conditional expressions are written using C/Java/Scala-style syntax:

if(⟨test⟩) ⟨expression⟩ **else** ⟨expression⟩

The two branches must have the same type, and this is the overall type of the conditional expression.

Functional applications can be written using parentheses, in the normal way, as in Figure 1. However, as in Haskell, the parentheses can be omitted for a function of one argument, where the actual parameter does not logically

need parentheses: for example, f 3, or g #A3. Function application associates to the left, and binds tighter than infix operators; for example, f g true + h #A represents ((f(g))(true)) + (h(#A)).

String literals are written inside double quotation marks, as in most other languages, e.g. "Hello". Special characters can be written using \, for example, "He said \"Hello\".\n".

A value of any type can be converted to a string using the toString function. Two strings can be concatenated using +. Further, a string can be concatenated with a non-string value using + (with the string on the left), whee the non-string value is implicitly converted to a string. For example

```
"Expected "+m+", found "+n+" in "+column+(row−#0)
```

produces a string such as "Expected 5, found 3 in #B4" (the subtraction of #0 converts the Row row into an Int, to avoid producing an extra "#"; see Section 4.2.3).

As noted in Section 4.1, the unit value is denoted ().

Tuple expressions are written inside parentheses, using commas, for example (x+y, 3.6, x>y). Tuples must contain at least two, and at most four components. When a function is applied to a tuple expression, an extra set of parentheses is required, for example f((x,y)) (since the expression f(x,y) would imply that f takes two arguments, rather than a single tuple argument). The functions get1, get2, get3, and get4 extract the corresponding component from a tuple; for example get2((2,4,6,8)) = 4. Tuples of the same type can be compared using the equality and order relations if the underlying types are themselves equality or ordered types; in the latter case, the ordering is lexicographic, so, for example (1, 2.0) < (1, 2.7).

A *block expression* consists of a sequence of declarations, followed by an expression, separated by semicolons or newlines, and enclosed in braces; for example,

```
{ def square(x: Int) = x*x; val y2 = square y; val z2 = square z; y2+z2 }
```

The declarations are evaluated in order; then the value of the final expression is the value of the block expression.

4.2.2 List expressions

List literals are written in square brackets, for example [1,2,3] or []. All the elements of the list must be of the same type. With the empty list, it is sometimes necessary to give the type, to help the type checker, for example []: List[Int].

List concatenation is written using ::, pronounced "cons". This operator takes an element on the left and a list on the right, and returns their con-

catenation; for example `3 :: [4,5] = [3,4,5]`. The function `head` returns the first element of a non-empty list; for example `head [3,4,5] = 3`. The function `tail` returns the list containing all except the first element of a non-empty list; for example `tail [3,4,5] = [4,5]`. The function `isEmpty` tests whether a list is empty.

Lists of consecutive `Int`s can be constructed using the infix operators **`to`** and **`until`**. The list `m to n` produces the list from `m` to `n` inclusive; for example, `3 to 5 = [3,4,5]`. The list `m until n` is similar, but does not include `n`; for example, `3 until 5 = [3,4]`. These operators can also be applied to row and column values; see below.

Lists of the same type can be compared using the equality and order relations if the underlying types are themselves equality or ordered types; in the latter case, the ordering is lexicographic, so, for example `[1, 2] < [1, 3]`.

List comprehensions provide a way to define a list, in a similar style to mathematical set comprehensions. The expression `[e | x <- xs]` generates the list containing the value of `e` for each `x` in `xs`. For example,

```
[x*x | x <- [3,4,5]]   =   [9,16,25].
```

A term of the form `x <- xs` is called a *generator*. A list comprehension may have multiple generators, separated by commas; an earlier generator is iterated over more slowly than a later generator. For example,

```
[x*y | x <- [2,3], y <- [4,5]]   =   [8,10,12,15].
```

A list comprehension may also have one or more boolean filters; a term is produced only for values of the variables that satisfy the filter. For example,

```
[x*y | x <- [2,3], y <- [4,5], x+y != 7]   =   [8,15].
```

A generator may perform pattern matching over tuples. A generator of the form `(x,y) <- pairs` expects `pairs` to be a list of pairs, and binds the names `x` and `y` to the two components of each pair in turn. For example,

```
[x*y | (x,y) <- [(2,4), (3,5)]]   =   [8,15].
```

This extends to tuples with more than two elements, and to nested tuples, in the obvious way.

4.2.3 Row, column, and cell expressions

Row and column literals are written using #, e.g. `#A`, `#B`, etc., and `#0`, `#1`, etc. A row or column value can have an `Int` added to it or subtracted from

it, with the obvious effect. For example, #C + 2 = #E, and #C − 2 = #A. Arithmetic underflow, e.g. from #C − 3, gives an error. Similarly, a row value can have another row value subtracted from it; and a column value can have another column value subtracted from it. For example, #5 − #2 = #E − #B = 3. Two row operators can be compared using the equality or order relations, with obvious meanings; for example, #3 < #5; two column values can similarly be compared.

Lists of row and column values can be formed using the **to** and **until** operators. For example #B **to** #D = [#B,#C,#D], and #2 **until** #4 = [#2,#3].

If c and r are a Row and Column, respectively, then Cell(c,r) is a *cell expression*: it represents the value in the corresponding cell of the spreadsheet. This can be abbreviated in the case of row and column literals; for example #D3 is shorthand for Cell(#D, #3).

The type checker needs to be able to deduce a type, or a set of possible types, for each cell expression. When the script is executed, the application checks if the corresponding cell in the spreadsheet has an expected type, and signals an error otherwise.

One option is to provide an explicit type for a cell expression. For example, #D3: Int gives the type Int to the cell expression.

Alternatively, the type can often be deduced automatically if a function is applied to the cell expression: this implies that its type must match the parameter type for the function. For example, in line 18 of Figure 1, the expression fact(Cell(In,r)) implies that the cell expression must have type Int, since fact takes a parameter of type Int.

Similarly, the type deduction can sometimes be made when a binary operator is applied to a cell expression. For example, consider the expression 1 + #A3; the type checker works from left to right, so it can deduce that the cell expression must have type Int. However, the explicit type is necessary in the similar expression #A3:Int + 1, because + is overloaded, and the type checker works from left to right.

Finally, a **match** expression allows the cell to have one of several different types, and for the value of the expression to depend on that type. An example was given on line 12 of Figure 1. In general, a **match** expression takes the form

⟨cell expression⟩ **match**{ ⟨cases⟩ }

where ⟨cases⟩ is a sequence of cases, separated by semicolons or newlines. Each case is of the form

case ⟨pattern⟩ => ⟨expression⟩

Each pattern takes one of the following forms.

- Empty, which matches an empty cell;
- ⟨name⟩: ⟨type⟩, where ⟨type⟩ is one of Int, Float, String, or Boolean; this matches a value of the given type, and binds the name to that value in the subsequent expression;
- _: ⟨type⟩, which is similar to the previous form, except no name is bound;
- _, which matches any type.

In a **match** expression, the cases are tried in the order given; the first case that matches is selected, and the corresponding expression is evaluated. If no case matches, an error is signalled. Within a **match** expression, all the expressions on the right-hand side of cases must have the same type.

4.3 Declarations

Declarations, of values and functions, may be made either at the top level of a script, or within a local block. Standard scoping rules apply.

A *value declaration* is written using the keyword **val**. Figure 1 contains several examples. Value declarations may pattern match against tuples. For example

```
val (x,y) = pair
```

This is particularly useful to unpack a tuple returned by a function.

A *function declaration* is written using the keyword **def**. Again, Figure 1 contains several examples. Each parameter of the function should be given a type.

A return type must be given for a recursive function, or if a forward reference is made to the function: this is because the type checker considers declarations in the order they appear, but can use claimed types of later functions. For example, the return type for f is necessary in the following, since the use of f precedes its definition:

```
val y = f(3); def f(x: Int): Int = x+1
```

However, no return type is necessary in the following, since the use of f is after its definition:

```
def f(x: Int) = x+1; val y = f(3)
```

A function can be defined that takes several vectors of parameters. For example:

```
def add(x: Int, y: Int)(z: Int) = x+y+z
val a = add(4,5) 6; val f = add(6,7); val b = f 8
```

Here f is a function of type Int => Int.

4.3.1 Polymorphism

Parametric polymorphism is supported in a style similar to as in Scala. Type parameters are introduced by enclosing them in square brackets after the name of a function, for example:

```
def apply[A,B](f: A => B, x: A): B = f(x)
```

This defines such a function for all types A and B. For example

```
def double(x: Float) = 2*x
val a = apply(head, [1,2,3]); val b = apply(double, 3.4)
```

Higher-order functions are supported in a style similar to Scala and Haskell. For example, we can define a function map such that map f applies f to each member of a list (of the approriate type):

```
def map[A,B](f: A => B)(xs: List[A]): List[B] =
  if(isEmpty xs) [] else f(head xs) :: map f (tail xs)
```

A type parameter A may be restricted to be a member of a type class TC using the notation A <: TC. For example, the following functions require their type parameter to be an equality type or ordered type, respectively.

```
def remove[A <: Eq](x: A)(xs: List[A]): List[A] =
  if(isEmpty xs) [] else if(head xs == x) tail xs else x :: remove x (tail xs)
def min[A <: Ord](x: A)(y: A) = if(x <= y) x else y
```

When a polymorphic function is called, concrete type parameters may be provided to instantiate the formal type parameters. This is necessary when the return type of a function call cannot be deduced from the types of the parameters. For example, in the following

```
def makeEmpty[A]() = []: List[A]; val xs = makeEmpty[Int]()
```

the concrete type parameter Int is necessary to deduce that xs is of type List[Int]. Likewise, it is sometimes necessary to provide a concrete type parameter so that the type checker can deduce the type of values read from cells. For example, in

```
def cells[A <: CellType](cols: List[Column])(row: Row): List[A] =
  [Cell(col,row): A | col <- cols]
val xs = cells[Float](#A to #C) #4
```

the concrete type parameter is necessary to deduce that the cells are expected to hold Floats.

4.3.2 Overloading

The language allows overloading of names of functions declared using **def**s (but not **val**s). For example,

```
def double(n: Int) = 2*n
def double(x: Float): 2*x
```

When an overloaded function is applied, the type checker needs to be able to determine which instance is intended, for example,

```
val m = double(3) // Int => Int instance.
```

In some cases, it is necessary to give an explicit type to the argument, particularly in the case of a cell read; for example,

```
val y = double(#A4: Float) // Float => Float instance.
```

To allow type deductions to be made, overloaded functions must have disjoint types for their first vectors of parameters, and may not be polymorphic.

4.4 Assertions

The basic form for an assertion is **assert**(⟨expression⟩), where the expression should evaluate to a boolean. When the script is executed, if that boolean is`false`, an error is generated.

A generalisation is **assert**(⟨expression⟩, ⟨expression⟩), where the second expression should evaluate to a string. If the assertion fails, that string is included in the error message.

4.5 Cell writes

The syntax for a cell write is ⟨cell-expression⟩ = ⟨expression⟩. Figure 1 contains examples where the cell expression on the left-hand side uses the `Cell` construction. The # construction can also be used on the left-hand side; for example,

```
#A4 = #A3:Int + 2
```

In most simple applications, cell writes occur only at the top level of a script. However, they may also appear within a function definition or a block expression. For example, the following code is equivalent to the **for** loop at line 18 of Figure 1.

```
def writeFacts(r: Row): Unit =
  if(r == firstEmpty) ()
  else{ Cell(out, r) = fact(Cell(In,r)); writeFacts(r+1) }
```

```
writeFacts(#0)
```

The function writeFacts(r) writes the factorials into rows r **until** firstEmpty. This is invoked via a function call; see Section 4.7.

Being able to preform cell writes within a function is useful in applications where the number of cells written depends upon earlier calculations. It allows a recursive function that keeps calculating and writing cells until a suitable end state is reached.

Recall that the application treats any attempt to write to a non-empty cell as an error.

4.6 for loops

The basic form of a **for** loop is

for (⟨name⟩ <− ⟨list⟩) ⟨statements⟩

where ⟨statements⟩ is either a single statement (a declaration, cell write, or **for** loop), or several statements enclosed in braces. For example,

```
for(r <− #0 to #5){
  val x = Cell(#A, r): Int; Cell(#B, r) = 2+x; Cell(#C, r) = 2*x
}
```

A **for** loop may have multiple generators, or boolean filters; for example,

```
for(r <− #0 to #5; c <− [#A, #C]; if r != #3)
  Cell(c+1,r) = Cell(c,r):Int + 3
```

A generator may pattern match against a tuple, for example,

```
for ((r,n) <− [(#1,2), (#2,4), (#3,6)])
  Cell(#A, r) = factorial(n)
```

Like cell writes, a **for** loop normally occurs just at the top level of a script, but may also occur within a function definition or a block expression.

4.7 Function calls

If a function returns the Unit type, a function call can be used as a statement. Section 4.5 contained an example.

4.8 Conditional statements

Statements can be executed conditionally, using an **IF** statement such as

```
IF(#A3:Int > 3){ #B3 = 4; #B4 = true } ELSE #B5 = false
```

The **ELSE** part is optional. Note that the keywords **IF** and **ELSE** are capitalised, to distinguish this statement from a conditional expression. Each branch may be a single statement or several statements enclosed in braces. Note that any declaration in a branch is local to that branch.

4.9 #include statements

A script may import the contents of another script via an #include statement, for example

```
#include "other-file.dir"
```

The name of the other file should be enclosed in quotation marks. There should be nothing following on the same line.

5 Multi-cell operations

The statements described in the previous section update at most one cell at a time, and did not allow writing to a cell that already holds a value. However, sometimes we need to go beyond this. The most common case is an operation to sort a block of cells, according to some criterion.

The language contains a built in function with signature

```
def sortBlockBy(columns: List[Column], rows: List[Row],
                before: (Row,Row) => Boolean): Unit
```

This sorts the block defined by columns and rows, according to the criterion defined by before: if before(r1,r2) gives true, then row r1 will be put before r2 in the sorted version (so before should correspond to an order relation).

Figure 5 illustrates this function. The script expects to find names in column #A from row #1, with corresponding Int scores in columns #B to #D. It calculates the total for each row, putting the results in column #E. However, we want ways to sort the data according to the total or by the name.

The operation sortByTotal() provides a way to sort the data into decreasing order of total score (column #E): the before function defines that row r1 will be placed before row r2 if it has a higher total score.

If we had placed the body of this operation at the top level of the script, i.e. not in any function or operation, then it would have been executed *every* time that the user entered a value in a cell. This would have been distracting for the user, because the row they were currently typing into could be moved.

```
#include "spreadsheet.dir"
val firstEmpty = firstEmptyInColumn(#A) // Find end of inputs.
val candidateRows = #1 until firstEmpty // Rows for candidates.
val columns = #A to #E // Columns in the table.
// Calculate totals.
for(r <- candidateRows) Cell(#E,r) = Cell(#B,r):Int + Cell(#C,r) + Cell(#D,r)
// Sorting operations.
operation sortByTotal() = {
  def before(r1: Row, r2: Row) = Cell(#E,r1): Int >= Cell(#E, r2)
  sortBlockBy(columns, candidateRows, before)
}
operation sortByName() = {
  def before(r1: Row, r2: Row) = Cell(#A,r1): String <= Cell(#A, r2)
  sortBlockBy(columns, candidateRows, before)
}
```

Figure 5: A script illustrating the sortByTotal function.

Instead, the keyword **operation** means that an item labelled "sortByTotal" is added to the "Operations" menu in the application. Selecting this item invokes the operation, and so sorts the data.

screenshot

The operation sortByName is similar, but sorts the data into decreasing order by the name field.